

-- Part 1: Creating database and importing each table in the database "saga"

create database if not exists saga;

use saga;

-- Creating staff table

```
create table staff (  
  staff_id varchar(10) primary key,  
  first_name varchar(10),  
  last_name varchar(10),  
  position varchar(15),  
  sal_per_hour decimal (10,2)  
);
```

select * from staff;

```
insert into staff(  
  staff_id, first_name, last_name, position, sal_per_hour)  
values  
( 'ST001', 'Emma', 'Johnson', 'Barista', 10.00),  
( 'ST002', 'Liam', 'Smith', 'Barista', 10.00),  
( 'ST003', 'Olivia', 'Williams', 'Barista', 10.00),  
( 'ST004', 'Noah', 'Brown', 'Barista', 10.00)  
;
```

select * from staff;

-- importing other tables directly using import option

```
select * from shift;  
select * from rota;  
select * from recipe;  
select * from orders;  
select * from items;  
select * from inventory;  
select * from ingredients;
```

show tables;

-- Part 2: Data Cleansing Steps

-- checking duplicates for the orders table

```
select row_id, count(row_id) from orders group by row_id having count(row_id) >1;
```

-- standardising data

```
select cust_name, trim(cust_name) from orders;
```

```
alter table orders
modify created_at datetime;
```

```
set sql_safe_updates=0;
```

```
update orders
set item_id = 'lt010'
where row_id= 45;
```

```
select * from orders;
```

-- checking column data types
describe orders;

-- checking missing or null values in orders table

```
select * from orders where order_id is null;
select * from orders where item_id is null;
select * from orders where cust_name is null;
select * from orders where in_or_out is null or in_or_out = '' ; -- 32 missing records
```

```
select order_id, in_or_out from orders where length(in_or_out) <> 0; -- 489 records
```

```
select order_id, in_or_out from orders where in_or_out <> '' and in_or_out not in('in','out') ;
-- 58 records neither missing nor null nor in,out
```

```
update orders
set in_or_out='n/a'
where in_or_out <> '' and in_or_out not in('in','out') ;
```

```
select in_or_out, count(in_or_out) from orders group by 1;
```

```
select * from orders;
```

-- checking duplicates for the items table

```
with items_cte as
(
select *,
row_number() over(
partition by item_id, sku, item_name, item_cat, item_size, item_price) as row_num
from items)
```

```
select * from items_cte where row_num>1;

-- standardising data

select item_name, trim(item_name) from items;

-- standardising item_price format in the items table:

alter table items
modify item_price DECIMAL(10, 2);

select * from items;

-- renaming column name 'sku' with 'recipe_id' in the items table

alter table items
change column sku recipe_id varchar(15) ;

select * from items;

-- checking duplicates for the recipe table

select row_id, count(row_id) from recipe group by row_id having count(row_id) >1;

-- making composite key using recipe_id and ing_id in the receipe table

create table recipe_new as
select *, trim(concat(recipe_id,'-',ing_id)) as receipe_ing_id
from recipe;

select * from recipe_new;

-- checking the unique records for 'receipe_ing_id' column in the recipe_new table

select count(*), count(distinct receipe_ing_id) from recipe_new;

-- Standardising ing_price format in the ingredients table:

alter table ingredients
modify ing_price DECIMAL(10, 2);

select * from ingredients;

-- checking the unique records for 'inv_id' and 'ing_id' column in the Inventory table

select count(*), count(distinct inv_id), count(distinct ing_id) from inventory;
```

```
select * from inventory;
```

```
-- The data for the staff, shift, and rota tables looks good
```

```
-- Part 3: Data Integration (Joins)
```

```
-- 1st join orders and items table
```

```
create table saga.orders_items as  
select a.*, b.recipe_id, b.item_name, b.item_cat, b.item_size, b.item_price  
from orders as a  
left join  
items as b  
on a.item_id = b.item_id;
```

```
select * from saga.orders_items; -- 521 records
```

```
select in_or_out, count(in_or_out) from saga.orders_items group by 1;
```

```
select * from orders_items where recipe_id is null; -- 54 records are null beacuse item_id  
It025-It028 aren't available in the items table
```

```
-- 2nd join items and recipe_new tables
```

```
create table saga.items_recipe as  
select a.*, b.row_id, b.ing_id, b.quantity, b.receipe_ing_id  
from items as a  
left join  
recipe_new as b  
on a.recipe_id = b.recipe_id;
```

```
select * from saga.items_recipe; -- 61 records
```

```
-- 3rd join recipe_new and ingredients tables
```

```
create table saga.recipe_ingredients as  
select a.*, b.ing_name, b.ing_weight, b.ing_meas, b.ing_price  
from recipe_new as a  
left join  
ingredients as b  
on a.ing_id = b.ing_id;
```

```
select * from saga.recipe_ingredients; -- 61 records
```

```
-- 4th join ingredients and inventory tables
```

```
create table saga.ingredients_inventory as
select a.*, b.inv_id, b.quantity
from ingredients as a
inner join
inventory as b
on a.ing_id = b.ing_id;
```

```
select * from saga.ingredients_inventory; -- 18 records
```

```
-- 5th join staff,rota and shift tables
```

```
create table saga.staff_rota_shift as
select concat(s.first_name,' ',s.last_name) as staff_full_name, s.staff_id, r.shift_id,
sh.day_of_week, sh.start_time, sh.end_time
from staff as s
left join
rota as r
on s.staff_id = r.staff_id
left join
shift as sh
on r.shift_id = sh.shift_id ;
```

```
select * from saga.staff_rota_shift; -- 18 records
```

```
-- 6th join orders, tems, recipes, and ingredients tables
```

```
-- Step 1: grouping ingredient price per recipe_id first
```

```
create table cost as
select r.recipe_id, sum(i.ing_price * r.quantity) AS total_receipe_cost
from
recipe_new as r
left join
ingredients as i
ON r.ing_id = i.ing_id
group by
r.recipe_id;
```

```
select * from cost;
```

```
-- Step 2: joining the aggregated recipe costs with orders and items table
```

```
create table saga.cost_analysis as
select o.order_id, o.created_at, o.cust_name,i.item_id, i.recipe_id, i.item_name, i.item_cat,
i.item_size, i.item_price as item_selling_price,
c.total_receipe_cost
```

```
from
orders as o
left join
items as i
on o.item_id = i.item_id
left join
cost as c
on i.recipe_id = c.recipe_id;
```

```
select * from saga.cost_analysis;
```

```
select * from saga.cost_analysis where item_id is null; -- 54 records
```

```
select item_id, sum(item_selling_price) as item_selling_price from saga.cost_analysis group
by 1;
```